

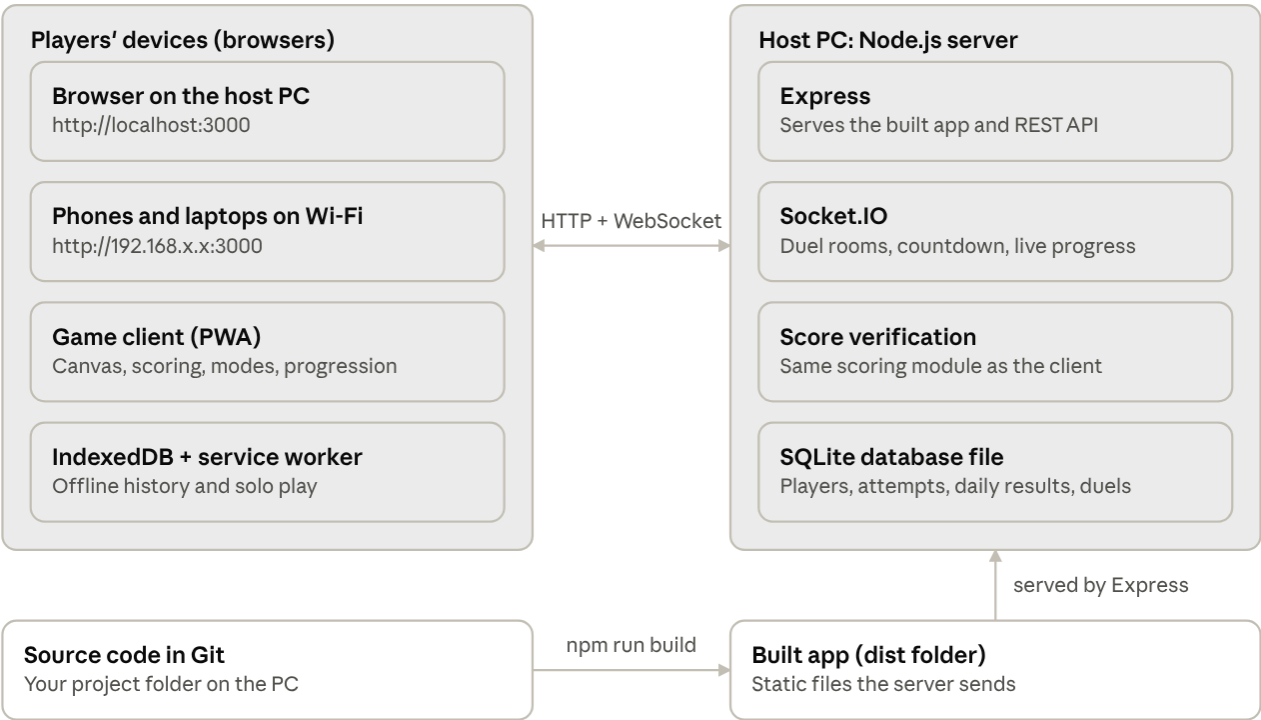
Perfect Circle — Technical Design Document

Prepared by: Manya Bansal · Sep 27, 2026 · Version 0.1

Version 0 (v0) of the game is a TypeScript PWA built with Vite and drawn on HTML canvas. A small Node.js server on your own PC serves it at `http://localhost:3000`, stores data in a SQLite file and runs duels over Socket.IO; there is no domain, cloud hosting or monthly cost.

Architecture

Everything runs on your PC; phones and laptops join over Wi-Fi



system architecture · device, backend, hosting, CI

Solo play, scoring and progression run in the browser, so they keep working if the Wi-Fi drops; the PC server is needed for logins, duels, leaderboards and verified scores.

Technology stack

Layer	Choice	Why
Language	TypeScript 5	Client and server share the same scoring code

Layer	Choice	Why
Build tool	Vite 5	Fast dev server, small production bundles
Rendering	HTML canvas 2D API	60 fps drawing on phones without a game engine
Input	Pointer Events API	One code path for mouse, touch and stylus
Charts	Chart.js 4	History graph on Profile
Browser storage	IndexedDB (via the idb library)	Offline history, badges and streak
PWA	vite-plugin-pwa (Workbox)	Manifest, icons, service worker, install prompt
Server	Node.js 20 + Express 4	Serves the built app and REST API on port 3000
Realtime	Socket.IO 4	Duel rooms and live progress over WebSocket, with automatic reconnect
Database	SQLite via better-sqlite3	One file on the PC; no database server to install
Security	bcryptjs, express-rate-limit	Hashed PINs, request limits
Process manager (optional)	pm2	Keeps the server running and restarts it after a crash
Local HTTPS (optional)	mkcert	Trusted certificate so phones can install the PWA
Unit tests	Vitest + Supertest	Client logic and server routes
End-to-end tests	Playwright	Real browser tests on Chrome, Safari (WebKit) and phone sizes
Version control	Git (GitHub optional)	History and backup; GitHub Actions can run tests

Project structure

```

perfect-circle/
  src/                client code that runs in the browser
    input/            pointer capture, resampling
    scoring/          circle.ts, polygon.ts, templates.ts, validate.ts (shared
with server)
    modes/            timed.ts, ink.ts, moving.ts
    render/           canvas drawing, colours, overlay
    daily/            seed.ts, share.ts
    progression/      streak.ts, badges.ts, history.ts
    duel/             room.ts, split-screen.ts
    data/             indexeddb.ts, sync.ts, api.ts
    screens/          home, setup, play, result, profile, settings
  server/
    index.ts          Express app, listens on port 3000
    routes/           login, attempts, daily, leaderboard
    duel.ts           Socket.IO rooms and round scoring
    db.ts             SQLite setup and queries
    schema.sql        table definitions
  data/
    perfect-circle.db  created on first run
  dist/              built client, served by Express
  tests/
    unit/            Vitest and Supertest
    e2e/             Playwright

```

Modules

Module	Responsibility	Key functions
input	Captures pointer points with timestamps; resamples to 128 points	capture(), resample(points, n)
scoring	Validates and scores strokes for each shape (used by client and server)	validate(stroke, opts), scoreCircle(), scorePolygon(stroke, shape)
modes	Adds timer, fading and moving-dot behaviour to the play loop	createTimer(), inkAlpha(age), dotPosition(t)
render	Draws the stroke, colours by error, draws the ideal overlay	drawStroke(), errorColour(e), drawIdeal()

Module	Responsibility	Key functions
daily	Picks today's challenge; builds the share text	dailyChallenge(date), shareText(result)
progression	Updates streak and badges; builds graph data	updateStreak(), checkBadges(), historySeries()
duel (client)	Joins rooms, sends progress and strokes; splits the screen	createRoom(), joinRoom(code), assignTouch(x)
data	Saves to IndexedDB, queues and syncs to the PC server	saveAttempt(), syncPending()
server/routes	Login, attempts, daily results, leaderboard endpoints	login(), postAttempts(), postDaily(), leaderboard()
server/duel	Socket.IO rooms, synced countdown, round scoring	onJoin(), startRound(), scoreRound()
server/db	Opens the SQLite file, creates tables, runs queries	openDb(), insertAttempt(), topScores()

Data model

Table (SQLite)	Key columns	Notes
players	id, nickname (unique), pin_hash, created_at	Created on first login
sessions	token, player_id, expires_at	30-day login cookie
attempts	id, client_id (unique), player_id, shape, mode, off_hand, score, created_at	Append-only; client_id stops duplicates on re-sync
daily_results	player_id, challenge_date, score, stroke (JSON, 128 points)	Primary key (player_id, challenge_date) enforces one attempt a day
badges_earned	player_id, badge_id, earned_at	Badge list itself lives in code
duel_rooms	code, host_id, guest_id, status, created_at	Rooms expire after 30 minutes
duel_rounds	room_code, round, shape, mode, host_score, guest_score, winner_id	Written only by the server after verifying both strokes

APIs and realtime events

- `POST /api/login`: nickname and PIN; creates the player on first use; sets a session cookie.
- `POST /api/attempts`: uploads queued attempts, each with a client_id so re-sending is safe. `GET /api/attempts`: returns the player's history to another device.
- `POST /api/daily`: takes the 128-point stroke; the server recalculates the score with the shared scoring module and stores it once per day.
- `GET /api/daily/:date/leaderboard`: top 100 nicknames and scores.
- Socket.IO, one room per duel code: events `join`, `ready`, `countdown` (with server start time), `progress` (0 to 1, sent 10 times a second), `stroke`, `round_result`, `match_result`.
- The daily challenge needs no API: `dailyChallenge(date)` gives the same answer on every device.

Security and privacy

- PINs are hashed with bcrypt; the plain PIN is never stored.
- Login uses an HTTP-only session cookie that lasts 30 days.
- Scores that others see (duels, daily leaderboard) are recalculated on the server; scores sent by a browser are ignored.
- Rate limit: 60 requests per player per minute.
- The server is reachable only on the local network; the firewall rule allows private networks only. No data leaves the PC.
- Backup: copy `data/perfect-circle.db` to another drive.

Offline and sync

- Every attempt is saved to IndexedDB first, then queued for upload.
- When the device can reach the PC server and the player is logged in, the queue uploads; the `client_id` on each attempt stops duplicates.
- Streak and badges are recalculated from attempts after sync, so phone and laptop agree.

Build and run on your PC

1. Install Node.js 20 LTS and Git on the PC.
2. Create the client: `npm create vite@latest perfect-circle -- --template vanilla-ts`, then `npm i chart.js idb socket.io-client` and `npm i -D vitest supertest @playwright/test vite-plugin-pwa tsx`.
3. Add the server packages: `npm i express socket.io better-sqlite3 bcryptjs express-rate-limit`, and create `server/index.ts`.
4. Move the current single-file game into the input, scoring and render modules; add unit tests for circle scoring.
5. Add square, triangle and star scoring with corner coverage, plus tests.
6. Add the four modes and the Setup screen.
7. Add IndexedDB storage, streaks, badges and the Chart.js history graph.
8. Add the daily challenge and share text.
9. Add the SQLite tables, login with nickname and PIN, and syncing of attempts.
10. Build duels with Socket.IO rooms and server-side score checks, then split-screen mode.
11. Turn on vite-plugin-pwa: app name, 192 px and 512 px icons, offline caching.

12. Build and start: `npm run build`, then `npm start`. The server serves `dist/` on port 3000 and listens on all network interfaces. Open `http://localhost:3000` on the PC.
13. Let other devices join: find the PC's IP address (`ipconfig` on Windows, the IPv4 Address line), allow Node.js through the firewall on private networks, then open `http://<that IP>:3000` on phones on the same Wi-Fi. Reserve a fixed IP for the PC in the router so the address does not change.
14. Optional, keep it running: `npm i -g pm2`, then `pm2 start npm --name perfect-circle -- start` and `pm2 save`.
15. Optional, phone install: create a local certificate with `mkcert` (`mkcert -install`, then `mkcert localhost 192.168.1.5`), start the server on HTTPS, and install the `mkcert` root certificate on each phone.
16. Commit to Git as you go; a GitHub repository with Actions can run Vitest and Playwright on every push.
17. Run business testing against the PC server, fix issues, and tag the release `v0.1`.